

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, classification_report, roc_auc_score, roc_curve
import matplotlib.pyplot as plt
```

```
from google.colab import files
uploaded = files.upload()
```



Choose Files

No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving data.csv to data.csv

```
data = pd.read_csv("data.csv")
```

```
data = data.drop(columns=["id", "Unnamed: 32"])
```

```
data["diagnosis"] = data["diagnosis"].map({'M': 1, 'B': 0})
```

```
X = data.drop(columns=["diagnosis"])
y = data["diagnosis"]
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
model = LogisticRegression()
model.fit(X_train_scaled, y_train)
```



LogisticRegression ⓘ ?
LogisticRegression()

```
y_pred = model.predict(X_test_scaled)
y_prob = model.predict_proba(X_test_scaled)[:, 1]
```

```
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

```
⇒ Confusion Matrix:  
[[70  1]  
 [ 2 41]]
```

```
print("\nClassification Report:")  
print(classification_report(y_test, y_pred))
```

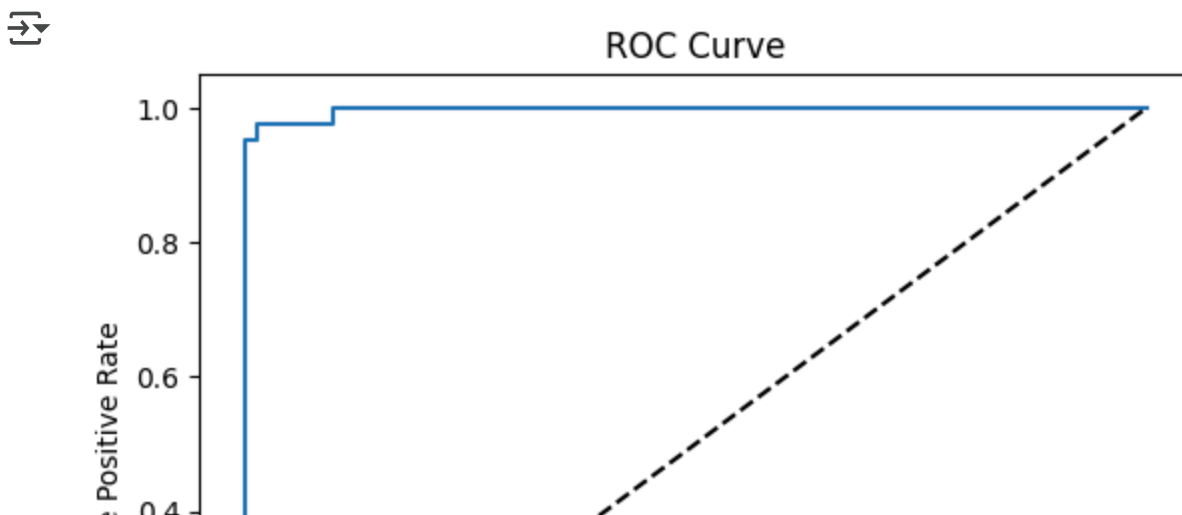
```
⇒  
Classification Report:  
              precision    recall  f1-score   support  
  
      0       0.97       0.99       0.98        71  
      1       0.98       0.95       0.96        43  
  
   accuracy       0.97  
  macro avg       0.97  
weighted avg       0.97
```

```
roc_score = roc_auc_score(y_test, y_prob)  
print("ROC-AUC Score:", roc_score)
```

```
⇒ ROC-AUC Score: 0.99737962659679
```

```
fpr, tpr, thresholds = roc_curve(y_test, y_prob)
```

```
plt.figure()  
plt.plot(fpr, tpr, label='Logistic Regression (area = %0.2f)' % roc_score)  
plt.plot([0, 1], [0, 1], 'k--') # Diagonal line  
plt.xlabel('False Positive Rate')  
plt.ylabel('True Positive Rate')  
plt.title('ROC Curve')  
plt.legend(loc="lower right")  
plt.show()
```



```

custom_threshold = 0.3
y_custom = (y_prob >= custom_threshold).astype(int)

print("Confusion Matrix with threshold 0.3:")
print(confusion_matrix(y_test, y_custom))

```

```

⇒ Confusion Matrix with threshold 0.3:
[[67  4]
 [ 1 42]]

```

```

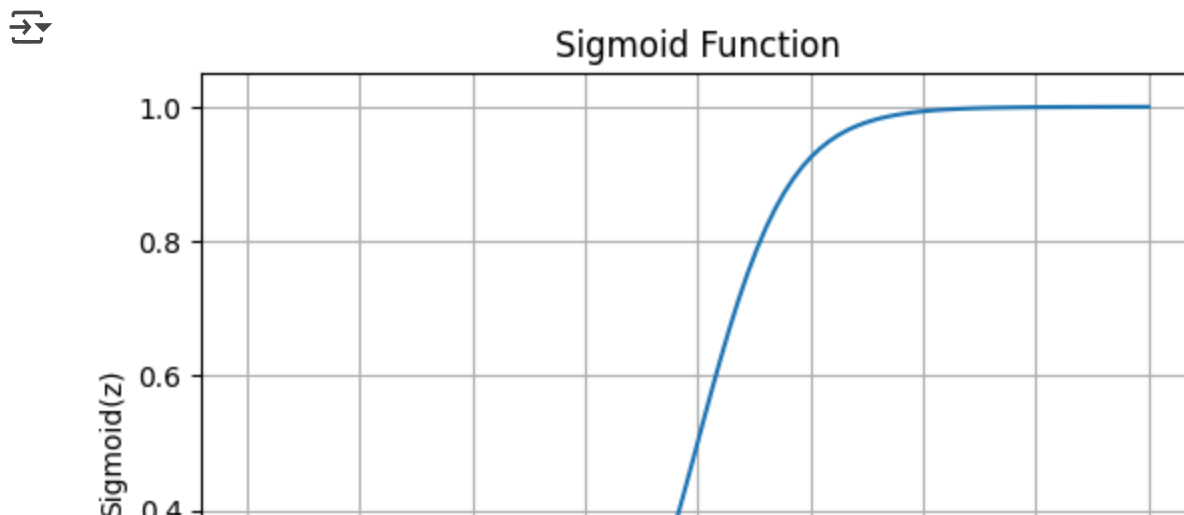
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

```

```

z = np.linspace(-10, 10, 100)
sig = sigmoid(z)
plt.plot(z, sig)
plt.title("Sigmoid Function")
plt.xlabel("z")
plt.ylabel("Sigmoid(z)")
plt.grid(True)
plt.show()

```



```
!ls -lha
```

```
⇒ total 140K  
drwxr-xr-x 1 root root 4.0K Apr 29 08:25 .  
drwxr-xr-x 1 root root 4.0K Apr 29 08:20 ..  
drwxr-xr-x 4 root root 4.0K Apr 25 17:43 .config  
-rw-r--r-- 1 root root 123K Apr 29 08:25 data.csv  
drwxr-xr-x 1 root root 4.0K Apr 25 17:44 sample_data
```

```
%%writefile UpdatedData.py
```

```
⇒ Writing UpdatedData.py
```

```
from google.colab import files  
files.download('UpdatedData.py')
```

```
⇒
```